



به نام خدا
دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر



درس شبکه‌های عصبی و یادگیری عمیق

تمرین شماره شش

طراحان تمرین:

سوال یک: سید مهدی موسوی

سوال دو: سعید رحیمی

مهلت ارسال: ۲۹ دی ۱۴۰۴

6	سوال ۱. بهبود عملکرد شبکه‌های VAE با افزایش ظرفیت توزیع پیشین
6	۱-۱. سوالات تئوری (۲۰ امتیاز)
6	۱-۱-۱. تابع هزینه مدل VAE (۵ امتیاز)
6	۱-۱-۲. تابع هزینه مدل جدید (۱۰ امتیاز)
6	۱-۱-۳. نوع بهبود حاصل از تابع هزینه (۵ امتیاز)
6	۱-۲. آشنایی با مجموعه داده (۵ امتیاز)
7	۱-۳. پیاده سازی در ابعاد نهان پایین (۱۰ امتیاز)
7	۱-۴. بررسی توزیع ابعاد نهان (۱۰ امتیاز)
7	۱-۵. پیاده سازی در ابعاد نهان بالا (۱۵ امتیاز)
7	۱-۶. بررسی posterior collapse (۱۰ امتیاز)
8	۱-۷. پیمایش فضای پنهان (۱۰ امتیاز)
8	۱-۸. نمایش شبه ورودی‌ها (۱۰ امتیاز)
8	۱-۹. محاسبه log likelihood (۱۰ امتیاز)
9	سوال ۲. افزایش داده با GAN برای بهبود طبقه بندی در شرایط کمبود داده
9	2-1. ساخت دیتالودر و انجام تنظیمات اولیه (5 نمره)
11	2-2. پیاده سازی Self Attention و Channel Attention (5 نمره)
11	2-3. لایه‌های کانولوشنی با Spectral Normalization برای Generator و Discriminator (5 نمره)
11	2-4. پیاده سازی Generator و Discriminator (15 نمره)
11	2-4-1. پیاده سازی بخش‌های اصلی معماری طبق مقاله
12	2-4-2. بخش شرطی سازی و Projection در Discriminator
12	2-5. پیاده سازی لاس‌ها (15 نمره)
12	2-5-1. پیاده سازی WGAN GP
13	2-5-2. سازگار کردن loss با حالت شرطی پروژه
13	2-5-3. پیاده سازی Drift
13	2-5-4. ترم L2 روی embedding مرتبط به کلاس‌ها در Discriminator
13	2-5-5. فرمول نهایی loss‌ها

14	6-2. ساخت بهینه سازها و انجام تنظیمات نهایی (10 نمره)
16	7-2. آموزش مدل (10 نمره)
17	8-2. نمایش نمودارها و خروجی مدل در طول آموزش و تحلیل آنها (10 نمره)
17	1-8-2. رسم نمودارها
18	2-8-2. ترسیم خروجی مدل در طول آموزش
18	3-8-2. سوالات تشریحی
19	9-2. تعریف کلاسیفایر و آموزش و ارزیابی آن با داده‌های اولیه (10 نمره)
19	10-2. آموزش و ارزیابی کلاسیفایر با داده‌های ساخته شده توسط مدل GAN آموزش دیده در مراحل قبل (10 نمره)
20	11-2. مقایسه عملکرد (10 نمره)
20	نکات مهم

جدول‌ها

10	جدول 1. پارامترهای اولیه مدل
15	جدول 2. پارامترهای مورد نیاز

سوال ۱. بهبود عملکرد شبکه‌های VAE با افزایش ظرفیت توزیع پیشین

مدل‌های VAE از مدل‌های ابتدایی تولید تصویر هستند. این مدل‌ها مانند شبکه‌های اتوانکودر از دو قسمت انکودر و دیکودر تشکیل شده‌اند اما برخلاف مدل‌های اتوانکودر هدف آنها بازسازی صرف نیست بلکه یادگیری ساختار فضای نهان و تولید نمونه جدید است. یکی از راه‌های افزایش کیفیت و قدرت این شبکه‌ها معرفی توزیع پیشین قابل یادگیری است در این مدل است که در این [مقاله](#) مطرح شده است. اگر به خاطر داشته باشید در مدل VAE ما سعی داریم توزیع پسین را به توزیع پیشین که معمولاً نرمال استاندارد است نزدیک کنیم (عبارت KL) اما در مقاله مطرح شده توزیع پیشین ما منعطف و قابل آموزش است که باعث میشود قدرت کلی مدل افزایش پیدا کند. توجه کنید که پیاده سازی معماری hierarchical این مقاله نیاز نیست و صرفاً به پیاده سازی عبارت لاس جدید خواهید پرداخت.

۱-۱. سوالات تئوری (۲۰ امتیاز)

۱-۱-۱. تابع هزینه مدل VAE (۵ امتیاز)

در مورد تابع هزینه مدل VAE توضیح دهید. به طور خاص نقش عبارت فاصله KL در این تابع هزینه به چه صورتی است.

۱-۱-۲. تابع هزینه مدل جدید (۱۰ امتیاز)

در مورد عبارت نرمال ساز مطرح شده در مقاله توضیح دهید. این عبارت چه تفاوتی با حالت قبلی دارد و چگونه به افزایش قدرت مدل می انجامد؟

۱-۱-۳. نوع بهبود حاصل از تابع هزینه (۵ امتیاز)

تصور کنید خروجی مدل VAE شما از استاندارد مورد نظر پایین تر است. نحوه استدلال برای انتخاب راه حل مناسب را بیان کنید. (افزایش قدرت شبکه، تغییر تابع هزینه، ..)

۱-۲. آشنایی با مجموعه داده (۵ امتیاز)

مجموعه داده‌ی مورد استفاده در این بخش Dynamic MNIST می باشد ابتدا توضیح مختصری درباره این مجموعه داده ارائه دهید و نحوه‌ی تولید آن را شرح دهید. سپس با ارائه استدلال مناسب، برتری Dynamic MNIST نسبت به مجموعه داده MNIST استاندارد را در چارچوب آموزش مدل‌های مولد، به ویژه VAE، توضیح دهید.

در ادامه، پس از تولید مجموعه داده، تعدادی از نمونه‌های داده را نمایش دهید.

۱-۳. پیاده سازی در ابعاد نهان پایین (۱۰ امتیاز)

در اولین قسمت از پیاده سازی نیاز است که یکبار مدل VAE ساده و بار دیگر مدل VAE را با تابع هزینه مطرح شده در مقاله آموزش دهید. برای هایپرپارامترهای شبکه و آموزش میتوانید از معماری و هایپرپارامترهای مطرح شده در شبکه استفاده کنید. (آموزش مدل در این قسمت و قسمت بعد حدود ۱۵ دقیقه زمان خواهد برد.)

در این قسمت اندازه ابعاد پنهان را برابر دو قرار دهید. بعد از آموزش برای هر یک از ترم‌های تابع هزینه و همچنین مقدار کلی تابع هزینه یک نمودار رسم کنید و در آنها منحنی‌های هر دو مدل را با هم قرار دهید و روند آموزش آنها را مقایسه کنید.

۱-۴. بررسی توزیع ابعاد نهان (۱۰ امتیاز)

پس از آموزش دو مدل VAE استاندارد و VAE با تابع هزینه‌ی پیشنهادی مقاله، تفاوت رفتار این دو مدل را می‌توان در فضای نهان بررسی کرد.

در این بخش، ابتدا نگاشت مجموعه‌ای از داده‌ها را در فضای نهان دوبعدی ترسیم کنید. در این نمایش، نقاط متناظر با نمونه‌های داده را بر اساس برجسب هر کلاس رنگ‌آمیزی کرده و الگوی پراکندگی آن‌ها را در فضای نهان برای هر یک از دو مدل مقایسه کنید.

سپس برای هر یک از دو مدل، توزیع پیشین و توزیع پسین را در فضای نهان به‌دست آورده و نمایش دهید. در ادامه، میزان همخوانی میان این دو توزیع را برای هر مدل بررسی کرده و تفاوت‌های مشاهده‌شده را تحلیل کنید. در نهایت، نتایج این دو آزمایش را تفسیر کرده و توضیح دهید که بهبود حاصل‌شده در مدل پیشنهادی چگونه و به چه شکلی در توزیع‌های فضای نهان قابل مشاهده است.

۱-۵. پیاده سازی در ابعاد نهان بالا (۱۵ امتیاز)

در این قسمت نیاز است که مدلی که در قسمت قبلی آموزش داده اید را با ابعاد پنهان بالاتر (برای پیشنهاد چهل) آموزش دهید. نمودارهای تابع هزینه را در این قسمت هم مانند قسمت قبل رسم کرده و سپس روند آموزش را هم بین این دو مدل و هم با حالت قبلی (بعد پنهان پایین) مقایسه کنید. همچنین برای هر کدام از این دو مدل که بعد نهان بالاتر دارند ۸ نمونه تولید کرده و کیفیت آنها را مقایسه کنید.

در ادامه به بررسی معیارهای مختلف برای این دو مدل دارای بعد نهان بالا می‌پردازیم.

۱-۶. بررسی posterior collapse (۱۰ امتیاز)

در مورد مشکل posterior collapse در شبکه‌های VAE توضیح دهید که این مشکل چیست چرا به وجود می‌آید. سپس به بررسی ابعاد پنهان بپردازید و فعال بودن ابعاد فضای پنهان را در دو مدل بررسی و مقایسه کنید. آیا مدل توانسته است بهبودی حاصل کند؟

۱- ۷. پیمایش فضای پنهان (۱۰ امتیاز)

از جمله آزمایش‌های مورد استفاده برای بررسی معنادار بودن فضای پنهان، میتوان به پیمایش در فضای نگاشت اشاره کرد. برای این منظور، ابتدا میانگین ابعاد نهان محاسبه می‌شود. سپس با به‌کارگیری تحلیل مؤلفه‌های اصلی (PCA) بر روی فضای نهان، جهت‌های اصلی تغییرات استخراج می‌گردد. در گام بعد، در قوی‌ترین جهت‌های به‌دست‌آمده از PCA حرکت کرده و نمونه‌های تولیدشده نمایش داده می‌شوند.

در پایان، نتایج حاصل مورد تحلیل قرار گرفته و توضیح داده می‌شود که این پیمایش‌ها چه اطلاعاتی درباره ساختار و معنادار بودن فضای نهان مدل ارائه می‌کنند.

۱- ۸. نمایش شبه ورودی‌ها (۱۰ امتیاز)

در این بخش، تعدادی از شبه‌ورودی‌ها (Pseudo-Inputs) را از دیکودر مدل عبور داده و نمونه‌های حاصل را نمایش دهید.

با استفاده از این نمایش‌ها، معناداری شبه‌ورودی‌ها و نقشی که در شکل‌گیری توزیع پیشین مدل ایفا می‌کنند را مورد بررسی قرار دهید.

۱- ۹. محاسبه log likelihood (۱۰ امتیاز)

معیار گزارش شده در مقاله را بررسی کنید. چرا به طور مستقیم این معیار در طول آموزش بهینه‌سازی نمیشود؟

تعدادی از روش‌های محاسبه لگاریتم لایکلیهود را گزارش کنید و سپس به کمک یکی از آنها لگاریتم لایکلیهود را برای این دو مدل محاسبه کنید. (برای هر دو مدل انتظار می‌رود اندازه این معیار کمتر از ۹۵ باشد.)

سوال ۲. افزایش داده با GAN برای بهبود طبقه بندی در شرایط کمبود داده

در بسیاری از مسائل هوش مصنوعی مشکل اصلی نبود دیتاست مناسب یا کم بودن تعداد نمونه‌های برچسب دار است. وقتی داده کم باشد، مدل‌های یادگیری عمیق به جای یاد گرفتن الگوهای واقعی، داده‌های آموزشی را حفظ می‌کنند و روی داده جدید افت عملکرد دارند. علاوه بر این، به دلیل کم بودن تنوع، مدل روی بعضی کلاس‌ها به صورت سیستماتیک خطا می‌دهد چون مرزهای آن کلاس‌ها را خوب یاد نگرفته است.

یکی از راه حل‌ها data augmentation است. هدف این کار فقط زیاد شدن تعداد داده نیست، بلکه این است که مدل مجبور شود ویژگی‌های عمومی تری یاد بگیرد و بهتر تعمیم بدهد. روش‌های ساده augmentation همیشه کافی نیستند، چون تنوع واقعی داده را پوشش نمی‌دهند. به همین دلیل از مدل‌های مولد استفاده می‌شود تا به جای دستکاری داده‌های موجود، خود توزیع داده را یاد بگیرند و نمونه‌های جدید بسازند.

در [این مقاله](#) که مبنای این پروژه است، از GAN برای تولید تصاویر مصنوعی استفاده شده است، با این هدف که این تصاویر نه فقط از نظر ظاهری باکیفیت باشند، بلکه وقتی وارد مرحله بعدی یعنی آموزش یک کلاسیفایر می‌شوند، واقعا باعث بهبود عملکرد آن شوند. در این پروژه ما به دنبال پیاده سازی این مقاله هستیم.

در مسیر پروژه، ابتدا یک مدل مولد برای هر کلاس تصویر تولید می‌کند. سپس این تصاویر به داده‌های آموزشی واقعی اضافه می‌شوند. در نهایت، یک کلاسیفایر دوبار از صفر آموزش داده می‌شود، یک بار فقط با داده واقعی و یک بار با داده واقعی به همراه داده مصنوعی. هر دو مدل روی یک مجموعه تست یکسان ارزیابی می‌شوند تا مشخص شود داده مصنوعی واقعا چه اثری روی دقت کلی و خطاهای هر کلاس داشته است.

نکته مهم: ما در این تمرین سعی میکنیم که بخشی از مقاله که مرتبط به طراحی مدل برپایه دیتاست MNIST است را پیاده سازی کنیم اما با این تفاوت که از دیتاست FashionMNIST استفاده میکنیم. همچنین تغییرات دیگری در معماری چند بخش از مدل انجام میدهم که در ادامه در بخش‌های مختلف توضیحات مربوطه به شما داد می‌شود.

2-1. ساخت دیتالودر و انجام تنظیمات اولیه (5 نمره)

یک پایپ لاین بسازید که FashionMNIST را لود کند، با کد زیر نرمال کند، سپس دیتاست را شافل کنید و آن را به دو Subset جدا تقسیم کنید: یکی برای GAN و یکی برای classifier. هر Subset باید دقیقا 5000 تصویر داشته باشد و بین دو Subset هیچ همپوشانی وجود نداشته باشد. تقسیم بندی باید متوازن به ازای هر کلاس انجام شود یعنی از هر کلاس دقیقا 500 تصویر در Subset مربوط به GAN و 500 تصویر دیگر در Subset مربوط به classifier قرار بگیرد. در نهایت به ازای هر کلاس 5 تصویر را در یک grid نمایش دهید. پارامترهایی که باید در این تمرین تنظیم کنید در جدول 1 آمده اند.

جدول 1. پارامترهای اولیه مدل

پارامتر	مقدار یا مقدار پیش فرض	توضیح کوتاه
seed	42	برای تکرارپذیری و ثابت بودن نمونه گیری
IMAGE_SIZE	28	اندازه تصویر
NC	1	تعداد کانالها
NZ	100	ابعاد نویز ورودی GAN
BATCH_SIZE	64	اندازه batch برای DataLoader
N_CLASSES	10	تعداد کلاسها
EMBED_DIM	100	بعد embedding کلاس
per_class	500	تعداد نمونه از هر کلاس برای هر Subset

برای نرمال سازی دیتاست از این کد استفاده کنید.

```
transform = transforms.Compose([
    transforms.ToTensor,()
    transforms.Normalize(mean=(0.5,), std=(0.5,))
])
```

برای دانلود دیتاست از این کد استفاده کنید.

```
from torchvision import datasets
train_dataset = datasets.FashionMNIST(
    root="data,"
    train=True,
    download=True,
    transform=transform
)
```

2-2. پیاده سازی Self Attention و Channel Attention (5 نمره)

در این بخش باید دو کلاس به نام‌های SelfAttention و ChannelAttention را مطابق مقاله پیاده سازی کنید. ورودی هر دو ماژول یک feature map با شکل (B, C, H, W) است و خروجی باید دقیقاً همین شکل را داشته باشد. برای SelfAttention طبق مقاله چهار نگاشت کانولوشن 1 در 1 به نام‌های f و g و h و v را بسازید، N را برابر $H \times W$ در نظر بگیرید و تنسورها را طوری reshape کنید که q و k بعد از نگاشت‌های مربوطه شکل (B, C', N) داشته باشند، h شکل (B, C, N) داشته باشد و ماتریس attention شکل (B, N, N) تولید شود. softmax را دقیقاً طوری اعمال کنید که با تعریف β در مقاله معادل باشد، سپس خروجی attention را به شکل (B, C, H, W) برگردانید و نگاشت v را بعد از جمع وزنی اعمال کنید، و در نهایت خروجی را به صورت $y = \gamma_0 + x$ بسازید که γ یک پارامتر قابل یادگیری با مقدار اولیه صفر است. برای ChannelAttention طبق مقاله ابتدا squeeze را با global average pooling انجام دهید تا هر کانال به یک عدد تبدیل شود، سپس excitation را با دو لایه fully connected و فعال ساز ReLU و در پایان sigmoid بسازید و وزن‌های کانالی به دست آمده را روی feature map اعمال کنید.

2-3. لایه‌های کانولوشنی با Spectral Normalization برای Discriminator و Generator (5 نمره)

در این بخش باید Spectral Normalization را همان طور که در مقاله استفاده شده، پیاده سازی کنید. بخش تئوری و این که SN دقیقاً چیست و چرا استفاده می شود را از مقاله یاد بگیرید. در کد، شما باید دو تابع کمکی بسازید که هدفشان این است ساخت لایه‌های کانولوشنی را استاندارد کنند و همیشه بعد از ساخت لایه، SN را روی همان لایه اعمال کنند و همان نسخه نرمال شده را برگردانند. یکی از این تابع‌ها باید Conv2d بسازد و یکی باید ConvTranspose2d بسازد. هر دو باید این ورودی‌ها را بگیرند: in_channels و out_channels و kernel_size و stride. هر دو باید پارامتر padding را هم داشته باشند که مقدار پیش فرض آن 0 است و پارامتر bias را هم داشته باشند. تابع ConvTranspose2d علاوه بر این‌ها باید پارامتر output_padding را هم داشته باشد که مقدار پیش فرض آن 0 است.

2-4. پیاده سازی Discriminator و Generator (15 نمره)

2-4-1. پیاده سازی بخش‌های اصلی معماری طبق مقاله

برای بخش‌های اصلی معماری شبکه که در مقاله آمده، خودتان باید به مقاله مراجعه کنید و همان معماری را دقیقاً استخراج و پیاده سازی کنید. منظور از این بخش‌های مقاله، تعداد لایه‌ها در Discriminator و Generator، ترتیب کلی بلوک‌ها، این که در کدام لایه‌ها Channel Attention و در کدام لایه‌ها Self Attention قرار می گیرد، مشخصات کانولوشن‌ها و کانولوشن‌های معکوس شامل تعداد کانال‌ها و اندازه کرنل و stride و padding، و همچنین استفاده از Spectral Normalization در لایه‌های کانولوشنی است.

2-4-2. بخش شرطی سازی و Projection Discriminator

در این پروژه پیاده سازی معماری مدل ما نسبت به معماری مقاله یک تفاوت مهم دارد. مدل ما نسبت به مدل مقاله شرطی یا conditional است. یعنی Generator و Discriminator علاوه بر ورودی‌های اصلی، برچسب کلاس (y) را هم دریافت می‌کنند و خروجی آن‌ها باید به y وابسته باشد. در Generator، ابتدا برچسب کلاس y که یک عدد است، با استفاده از embedding به یک بردار عددی 100 بعدی تبدیل می‌شود. این بردار نقش اطلاعات مربوط به کلاس را بازی می‌کند. سپس این بردار در کنار بردار نویز تصادفی z با بعد 100 قرار داده می‌شود، به این معنا که کانال‌های مربوط به نویز و کانال‌های مربوط به کلاس بدون ترکیب شدن، فقط پشت سر هم قرار می‌گیرند و با هم ورودی نهایی Generator را می‌سازند.

برای Discriminator از روش Projection Discriminator استفاده کنید. در این روش، شبکه تصویر ورودی را پردازش می‌کند و در لایه‌های پایانی به یک نمایش عددی فشرده از تصویر می‌رسد که به آن بردار ویژگی تصویر گفته می‌شود. این بردار که در ادبیات یادگیری عمیق با نام feature vector یا feature representation شناخته می‌شود، خروجی لایه‌های کانولوشنی Discriminator است که اطلاعات مهم تصویر را به صورت یک بردار خلاصه در خود نگه می‌دارد.

به طور همزمان، برای هر برچسب کلاس y نیز یک بردار embedding یاد گرفته می‌شود که آن هم یک نمایش عددی از کلاس در فضای شبکه است. سپس بین بردار ویژگی تصویر (feature vector) و بردار embedding مربوط به کلاس y یک ضرب داخلی محاسبه می‌شود. نتیجه این ضرب داخلی یک عدد است که میزان سازگاری تصویر با آن کلاس را نشان می‌دهد. Discriminator علاوه بر این امتیاز شرطی، یک امتیاز کلی نیز برای واقعی یا جعلی بودن تصویر محاسبه می‌کند و خروجی نهایی آن برابر است با جمع این دو مقدار. در این پروژه، طول بردار embedding کلاس‌ها باید 512 انتخاب شود، چون بردار ویژگی تصویر نیز 512 بعدی است و برابر بودن این دو طول باعث می‌شود این محاسبه به صورت مستقیم و معنی دار انجام شود.

در صورتی که نیاز به مطالعه در مورد روش Projection Discriminator دارید می‌توانید از [این لینک](#) یا [این لینک](#) کمک بگیرید.

2-5. پیاده سازی لاس‌ها (15 نمره)

2-5-1. پیاده سازی WGAN GP

در این بخش باید تابع هزینه WGAN GP را مطابق مقاله پیاده سازی کنید. یعنی ترم Wasserstein را بر اساس میانگین امتیازهای Discriminator روی نمونه‌های واقعی و جعلی بسازید و همچنین Gradient Penalty را طبق تعریف مقاله محاسبه کنید. Gradient Penalty باید با نمونه‌گیری interpolate بین real و fake ساخته شود، گرادیان خروجی Discriminator نسبت به همین interpolate محاسبه شود و پناستی گرفته شود و ضریب این ترم باید دقیقاً برابر 10 باشد.

2-5-2. سازگار کردن loss با حالت شرطی پروژه

چون Discriminator در این پروژه شرطی است، در تمام جاهایی که Discriminator فراخوانی می شود باید labels هم به آن داده شود. این شامل محاسبه امتیاز real، محاسبه امتیاز fake، و محاسبه امتیاز روی interpolates در Gradient Penalty است. برای Gradient Penalty در این پروژه باید از همان برجسب‌های real استفاده کنید.

2-5-3. پیاده سازی Drift

علاوه بر اجزای مقاله، یک drift penalty هم باید اضافه کنید که از میانگین مربع امتیازهای real و fake ساخته می شود و فقط داخل loss دیسکریمیناتور جمع می شود. ضریب این ترم باید دقیقاً برابر 0.001 باشد. این ترم روی loss ژنراتور اعمال نمی شود.

2-5-4. ترم L2 روی embedding مرتبط به کلاس‌ها در Discriminator

چون Discriminator در این پروژه یک embedding مرتبط به کلاس‌ها برای Projection Discriminator دارد، باید یک regularization از نوع L2 روی وزن‌های embedding مرتبط به کلاس‌ها اضافه کنید. این ترم برابر میانگین مربع وزن‌های embedding است و فقط داخل loss دیسکریمیناتور جمع می شود. ضریب این ترم باید دقیقاً برابر 0.001 باشد.

2-5-5. فرمول نهایی loss‌ها

loss نهایی Discriminator باید برابر باشد با ترم Wasserstein به علاوه 10 ضرب در Gradient Penalty به علاوه drift penalty با ضریب 0.001 به علاوه L2 روی embedding با ضریب 0.001. loss نهایی Generator باید فقط منفی میانگین امتیاز Discriminator روی نمونه‌های جعلی با برجسب‌هایشان باشد و هیچ کدام از drift و L2 embedding وارد loss ژنراتور نمی شود. برای درک بهتر نحوه پیاده سازی توابع لاس از سودوکد زیر میتوانید کمک بگیرید.

PSEUDOCODE: Conditional WGAN GP losses

Constants

```
LAMBDA_GP      = 10.0
LAMBDA_DRIFT    = 1e-3
LAMBDA_EMB_L2   = 1e-3
```

Assumptions

```
D(image, label) returns one scalar score per sample (shape: [batch])
G(noise, label) returns a generated image (shape: [batch, C, H, W])
```

For the critic update, fake images are generated using the same labels as the real batch

For the generator update, labels can be sampled randomly

```
Function: GRADIENT_PENALTY(D, real_images, fake_images, labels)
    batch_size = real_images.size(0)
    epsilon = Uniform(0, 1) with shape [batch_size, 1, 1, 1]
    interpolated = epsilon * real_images + (1 - epsilon) * fake_images
    interpolated.requires_grad = True
    scores = D(interpolated, labels) # shape [batch_size]
    gradients = gradient of scores w.r.t. interpolated # shape [batch_size,
C, H, W]
    gradients = flatten gradients to shape [batch_size, C*H*W]
    grad_norm = L2_norm(gradients, axis=1) # shape [batch_size]
    gp = mean( (grad_norm - 1)^2 )
    return gp

Function: DISCRIMINATOR_LOSS(D, real_images, real_labels, fake_images, fake_labels)
    real_scores = D(real_images, real_labels) # shape [batch]
    fake_scores = D(fake_images, fake_labels) # shape [batch]
    wasserstein_term = mean(fake_scores) minus mean(real_scores)
    gp = GRADIENT_PENALTY(D, real_images, fake_images, real_labels)
    drift = 0.5 * mean(real_scores^2) + 0.5 * mean(fake_scores^2)
    emb_l2 = 0
    if D has a class embedding matrix (for example D.class_emb.weight):
        emb_l2 = mean( D.class_emb.weight^2 )

    total_loss = wasserstein_term
                + (LAMBDA_GP * gp)
                + (LAMBDA_DRIFT * drift)
                + (LAMBDA_EMB_L2 * emb_l2)

    return total_loss

Function: GENERATOR_LOSS(D, fake_images, fake_labels)
    fake_scores = D(fake_images, fake_labels) # shape [batch]
    loss_g = minus mean(fake_scores)
    return loss_g
```

2-6. ساخت بهینه سازها و انجام تنظیمات نهایی (10 نمره)

در این بخش باید دقیقا از مقادیر و تنظیماتی که در همین قسمت به شما داده می شود استفاده کنید و به مقاله رجوع نکنید، چون این قسمت نسبت به مقاله تغییر داده شده است.

یک تابع مقداردهی اولیه بنویسید و روی هر دو مدل اعمال کنید. برای لایه‌های Conv2d، ConvTranspose2d، Linear و Embedding وزن‌ها را با نرمال با میانگین 0 و انحراف معیار 0.02 مقداردهی کنید و bias را صفر کنید. برای BatchNorm2d وزن را با نرمال با میانگین 1 و انحراف معیار 0.02 مقداردهی کنید و bias را صفر بگذارید. اگر لایه دارای spectral normalization بود باید وزن اصلی آن که داخل weight_orig نگه داشته می شود مقداردهی اولیه شود.

پارامترهای هر مدل را به دو گروه تقسیم کنید. گروه اول همه پارامترها به جز embeddingها است و باید weight decay برابر صفر داشته باشد. گروه دوم فقط پارامترهای embedding است و باید weight decay غیر صفر داشته باشد. در Discriminator فقط embedding کلاس‌ها و در Generator نیز embedding برجسب‌ها داخل این گروه قرار می گیرند.

برای Generator و Discriminator بهینه ساز Adam بسازید. نرخ یادگیری Generator و Discriminator متفاوت است و betas باید دقیقاً مقدارهای داده شده باشد. روی هر optimizer یک scheduler نمایی بسازید که نرخ یادگیری را با gamma کاهش دهد و این کاهش را با فاصله گام مشخص اعمال کنید.

در آخر باید سه پیاده‌سازی دیگر هم انجام دهید تا بتوانید داده‌های متنوع برای آموزش تولید کنید و هم بتوانید پیشرفت مدل را در طول زمان منصفانه و قابل مقایسه ببینید. اول، یک ساز و کار تولید نویز تصادفی لازم دارید چون ورودی اصلی ژنراتور یک بردار تصادفی است و بدون نویز، خروجی‌ها تنوع واقعی ندارند. این نویز باید از توزیع نرمال استاندارد تولید شود و هر بار که آموزش انجام می شود بتوانید به تعداد batch_size نمونه نویز تولید کنید و شکل خروجی دقیقاً (batch_size, NZ) باشد. دوم، یک مجموعه نویز ثابت لازم دارید که فقط یک بار در ابتدای کار ساخته می شود و تا آخر آموزش تغییر نمی کند. دلیلش این است که می‌خواهیم نمونه‌های خروجی مدل را در طول آموزش ذخیره کنیم، اگر ورودی هر بار عوض شود نمی توانید تشخیص دهید تغییر خروجی به خاطر بهتر شدن مدل بوده یا فقط به خاطر عوض شدن ورودی. پس fixed_noise یک معیار ثابت برای مقایسه زمانی است. اندازه fixed_noise باید به اندازه ای باشد که برای هر کلاس چند نمونه تولید کنید. سوم، چون مدل شرطی است و خروجی به لیبیل وابسته است، باید یک بردار لیبیل ثابت هم بسازید تا هر بار که نمونه تولید می کنید دقیقاً همان کلاس‌ها و همان تعداد تکرار را داشته باشید و بتوانید کیفیت را برای تک تک کلاس‌ها مقایسه کنید. fixed_labels باید شامل همه کلاس‌ها باشد و هر کلاس دقیقاً FIXED_PER_CLASS بار تکرار شود تا مجموعه ارزیابی استاندارد ساخته شود. در طول آموزش، هر زمان که می خواهید خروجی نمونه گیری و ذخیره کنید، باید از fixed_noise و fixed_labels استفاده کنید تا نمونه‌های ذخیره شده قابل مقایسه باشند.

جدول 2. پارامترهای مورد نیاز

توضیح	مقدار	مؤلفه
نرخ یادگیری بهینه ساز ژنراتور	0.0002	LR_G
نرخ یادگیری بهینه ساز دیسکریمیناتور	0.0004	LR_D
β_1, β_2	(0.0, 0.9)	Adam betas

Weight init conv/linear/emb	Normal(mean=0.0, std=0.02)	مقداردهی اولیه وزن‌های لایه‌های کانولوشن، کانولوشن معکوس، تمام متصل و embedding
Weight init BatchNorm weight	Normal(mean=1.0, std=0.02)	مقداردهی اولیه وزن‌های لایه‌های نرمال سازی دسته‌ای
All bias init	0.0	مقداردهی اولیه تمام بایاس‌ها
If SpectralNorm exists	init weight_orig	اگر روی یک لایه SpectralNorm دارید، باید وزن اصلی همان لایه را مقداردهی کنید
Embedding weight_decay	0.001	میزان weight decay فقط برای پارامترهای embedding
Other weight_decay	0.0	میزان weight decay برای همه پارامترهای غیر embedding
Scheduler type	ExponentialLR	نوع زمان بندی کاهش نرخ یادگیری
LR decay gamma	0.95	ضریب کاهش نرخ یادگیری در هر بار اعمال scheduler
LR decay every steps	100	هر چند گام یک بار scheduler اجرا شود
N_CRITIC	5	تعداد به روزرسانی‌های دیسکریمیناتور به ازای هر به روزرسانی ژنراتور
RUN_STEPS	4000	تعداد کل گام‌های آموزش
FIXED_PER_CLASS	8	تعداد نمونه ثابت برای هر کلاس در ارزیابی دوره ای
Noise distribution	Normal(0,1)	توزیع نویز ورودی ژنراتور

2-7. آموزش مدل (10 نمره)

در این بخش باید یک حلقه آموزش بنویسید که در هر گام دقیقاً این ترتیب را رعایت کند. در هر step، ابتدا چند بار پشت سر هم Discriminator را آپدیت می‌کنید و بعد یک بار Generator را آپدیت می‌کنید. هدف این است که Discriminator در هر گام کمی جلوتر از Generator باشد تا تخمین فاصله Wasserstein و گرادیان

پنالتی پایدارتر شود. برای آپدیت‌های Discriminator، هر بار یک batch تصویر واقعی و لیبیل همان batch را از دیتالودر می‌گیرید و به device منتقل می‌کنید. سپس به همان تعداد نویز تولید می‌کنید و با استفاده از ژنراتور و همان لیبیل‌های واقعی، تصویر جعلی می‌سازید. این تصاویر جعلی برای آپدیت Discriminator باید از گراف محاسباتی جدا شوند تا گرادیان به ژنراتور برنگردد. بعد loss دیسکریمیناتور را محاسبه می‌کنید که شامل ترم Wasserstein به علاوه gradient penalty و همچنین ترم drift و L2 روی embedding کلاس است، سپس گرادیان می‌گیرید و optimizer دیسکریمیناتور را step می‌کنید. این چرخه را دقیقاً N_CRITIC بار تکرار می‌کنید.

بعد از تمام شدن N_CRITIC آپدیت دیسکریمیناتور، نوبت به یک آپدیت ژنراتور می‌رسد. برای آپدیت ژنراتور، یک batch نویز جدید می‌سازید و یک batch لیبیل تصادفی در بازه کلاس‌ها تولید می‌کنید تا ژنراتور یاد بگیرد برای همه کلاس‌ها تصویر تولید کند. سپس تصاویر جعلی را تولید می‌کنید و آن‌ها را همراه با همان لیبیل‌ها به Discriminator می‌دهید تا برای هر تصویر یک امتیاز خام بگیرد. در WGAN این امتیاز “احتمال” نیست و فقط یک نمره است، و هدف ژنراتور این است که میانگین این امتیاز را برای تصاویر جعلی تا حد ممکن بالا ببرد تا تصاویرش واقعی‌تر به نظر برسند. چون در آموزش همیشه یک تابع را کمینه می‌کنیم، loss ژنراتور را برابر منفی میانگین امتیاز Discriminator روی همین تصاویر جعلی با همین لیبیل‌ها می‌گیرید. در نتیجه با کمینه کردن این loss، ژنراتور عملاً میانگین امتیاز تصاویر جعلی را بیشینه می‌کند. بعد گرادیان می‌گیرید و optimizer ژنراتور را step می‌کنید.

در طول آموزش، باید یک برنامه کاهش نرخ یادگیری داشته باشید و آن را هر LR_DECAY_EVERY گام یک بار اعمال کنید تا نرخ یادگیری به صورت نمایی کاهش یابد. همچنین برای جلوگیری از مشکل تمام شدن دیتالودر، اگر iterator به انتها رسید باید دوباره از اول ساخته شود تا آموزش بدون توقف ادامه پیدا کند. در نهایت برای اینکه اگر اجرا قطع شد یا خطا رخ داد پیشرفت از دست نرود، توصیه می‌شود هر 100 گام یک بار وزن‌ها و آمارهای مختلف مدل را ذخیره کنید تا بتوانید آموزش را از همان نقطه ادامه دهید و همچنین در مراحل بعدی برای تحلیل از آنها استفاده کنید.

2-8. نمایش نمودارها و خروجی مدل در طول آموزش و تحلیل آنها (10 نمره)

2-8-1. رسم نمودارها

این چهار نمودار را رسم کنید. در همه نمودارها محور افقی step است. محور عمودی مقدار کمیت‌هایی است که روی همان نمودار رسم می‌کنید.

نمودار ۱: سه منحنی را روی یک نمودار رسم کنید. loss ژنراتور، loss دیسکریمیناتور، و تخمین فاصله Wasserstein

نمودار ۲: سه منحنی را روی یک نمودار رسم کنید. تخمین فاصله Wasserstein، گرادیان پنالتی، و drift penalty

نمودار ۳: سه منحنی را روی یک نمودار رسم کنید. امتیاز میانگین داده واقعی، امتیاز میانگین داده جعلی، و تخمین فاصله Wasserstein

نمودار ۴: دو منحنی را روی یک نمودار رسم کنید. نرخ یادگیری ژنراتور و نرخ یادگیری دیسکریمیناتور.

2-8-2. ترسیم خروجی مدل در طول آموزش

در گزارش، علاوه بر نمودارها باید خروجی‌های تصویری ژنراتور را هم به صورت گرید قرار دهید. گریدها را فقط برای stepهای 0، 1000، 2000، 3000 و 4000 تولید و در گزارش بگذارید. برای اینکه مقایسه منصفانه باشد، در همه این stepها باید دقیقا از یک نویز ثابت و یک لیبیل ثابت استفاده کنید تا تنها چیزی که تغییر می‌کند وزن‌های مدل در طول آموزش باشد (در مراحل قبل توضیحات آن به شما داده شده است). چیدمان گرید باید این طور باشد که همه کلاس‌ها در آن حضور داشته باشند و برای هر کلاس دقیقا 8 نمونه نمایش داده شود. بهترین حالت این است که گرید را با 8 ستون بسازید تا هر ردیف مربوط به یک کلاس باشد و 8 تصویر آن کلاس کنار هم دیده شود. در نهایت این 5 گرید را به ترتیب زمانی در گزارش قرار دهید و زیر هر کدام step مربوطه را بنویسید.

2-8-3. سوالات تشریحی

حالا به سوال‌های زیر را کوتاه ولی دقیق جواب دهید و حتما به نمودارها استناد کنید.

الف) چه زمانی تخمین فاصله Wasserstein از نظر مفهومی قابل اعتماد نیست؟ فقط با تکیه بر رفتار گرادیان پناستی توضیح دهید و بگویید چرا.

ب) از روی تغییرات امتیاز جعلی و تخمین فاصله، چطور می‌فهمید ژنراتور واقعا بهتر شده و فقط مقیاس امتیازها عوض نشده؟

پ) drift penalty دقیقا جلوی چه مشکلی را می‌گیرد؟ یک نشانه واضح در نمودار امتیاز واقعی و جعلی بگویید که اگر drift نبود احتمال داشت ببینید.

ت) در بازه‌هایی که تخمین فاصله بالا و پایین می‌شود، سهم اصلی بیشتر از تغییر امتیاز واقعی است یا امتیاز جعلی؟ نتیجه این را درباره اینکه کدام طرف بازی غالب بوده توضیح دهید.

ث) اگر تخمین فاصله بزرگ باشد ولی کیفیت بصری نمونه‌ها هنوز بد باشد، از نظر مفهومی دو دلیل محتمل بگویید و بگویید در همین نمودارها دنبال چه نشانه‌ای می‌گردید تا تشخیص دهید کدام دلیل محتمل تر است.

ج) بعد از کاهش نرخ یادگیری، انتظار دارید کدام نوع ناپایداری کمتر شود؟ آیا در نمودارهای فاصله و پناستی‌ها اثرش را می‌بینید؟ اگر نه، یک دلیل مرتبط با ماهیت بازی GAN بیان کنید.

2-9. تعریف کلاسیفایر و آموزش و ارزیابی آن با داده‌های اولیه (10 نمره)

در این بخش کد کلاسیفایر را که در زیر به شما داده شده دقیقاً همان طور که هست استفاده کنید و فقط آن را آموزش و ارزیابی کنید. مدل را روی دیتاست کلاسیفایر که در ابتدای کار درست کردید آموزش دهید و آن را به دو بخش train و test با اندازه‌های ۳۵۰۰ و ۱۵۰۰ تقسیم کنید. آموزش را با Adam انجام دهید، learning rate برابر ۰.۰۰۱ و تعداد epoch برابر ۱۵ و loss از نوع Cross Entropy باشد. در گزارش، مقادیر نهایی Train loss و Test loss و Train accuracy و Test accuracy را بنویسید و در پایان دو نمودار رسم کنید، یکی Loss by epoch برای train و test و یکی Accuracy by epoch برای آن دو.

```
class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=0)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.avg_pool2d(x, kernel_size=2, stride=2)
        x = F.relu(self.conv2(x))
        x = F.avg_pool2d(x, kernel_size=2, stride=2)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

2-10. آموزش و ارزیابی کلاسیفایر با داده‌های ساخته شده توسط مدل GAN آموزش دیده در مراحل قبل (10 نمره)

در این بخش باید یک دیتاست آموزشی جدید بسازید که ترکیبی از داده‌های واقعی و داده‌های مصنوعی باشد. نکته مهم این است که مجموعه test و train را تغییر ندهید و همان test set قبلی با ۱۵۰۰ تصویر و train set قبلی با دقیقاً همان ۳۵۰۰ تصویر را نگه دارید تا در ادامه مقایسه منصفانه باشد. حالا باید برای هر کلاس دقیقاً ۱۴۰۰ تصویر مصنوعی توسط مدل GAN که قبلاً آموزش دادید تولید کنید و به داده‌های آموزشی train set اضافه

کنید. یعنی در مجموع ۱۴۰۰۰ تصویر مصنوعی دارید و وقتی آن را به ۳۵۰۰ تصویر واقعی اضافه کنید، اندازه دیتاست آموزش جدید باید ۱۷۵۰۰ تصویر شود.

سپس کلاسیفایر را از صفر بسازید و از ابتدا با همان تنظیمات قبلی آموزش دهید. در پایان روی همان ۱۵۰۰ تصویر تست ارزیابی کنید. در گزارش، مقادیر نهایی Train loss و Train accuracy و Test loss و Test accuracy را بنویسید و در پایان دو نمودار رسم کنید، یکی Loss by epoch برای train و test و یکی Accuracy by epoch برای آن دو.

2-11. مقایسه عملکرد (10 نمره)

برای هر دو حالت، این چهار خروجی را روی تست محاسبه و در گزارش کنار هم قرار دهید: Test accuracy، Test macro F1، Confusion matrix، و نمودار Row normalized به صورت Row normalized، و نمودار Per class F1. سپس یک جمع بندی کوتاه بنویسید که آیا داده مصنوعی باعث بهبود کلی شده یا نه. در این جمع بندی حتما عدد دقیق بهبود را هم بنویسید، یعنی اختلاف Test accuracy را به صورت درصد گزارش کنید و مشخص کنید چند درصد بهتر شده است. همچنین اگر بهبود رخ داده مشخص کنید کدام کلاس ها بیشترین سود را برده اند و کدام جفت کلاس ها کمتر با هم اشتباه گرفته می شوند.

نکات مهم

- نتایج نهایی تولید شده توسط مدل Gan شما باید در حد شکل ۱ یا از آن بهتر باشد.
- دقت کلاسیفایر شما باید در حالت بدون داده های تولیدی مصنوعی بر روی داده های تست حدود 79 درصد و در حالت با دیتاست دارای داده های مصنوعی تولید شده حدود 82 درصد یا بیشتر باشد. در نهایت شما باید موفق شوید دقت کلاسیفایر را با داده های تولید شده 3 درصد یا بیشتر بهبود دهید. در صورتی که دقت کلاسیفایرهای شما از این اعداد پایین تر باشد مشخص کننده این است که مراحل قبل را به درستی انجام نداده اید.



شکل 1. نمونه خروجی مدل GAN آموزش دیده